

XOR на свету

почему один слой не может — и как обратная связь это чинит

Подробный разбор: от того, почему одно усреднённое число не способно вычислить XOR, до пошаговой двухслойной схемы через обратную связь экран → камера.

Проект «Светоч» — оптический нейрокомпьютер на телефоне, зеркале и фронтальной камере.

Эта заметка отвечает на два частых вопроса по эксперименту «Логические вентили» (stage4_logic):

1. Камера интегрирует по всему кадру — или есть варианты?
2. Можно ли через обратную связь экран → камера проэмулировать двухслойную сеть и получить XOR?

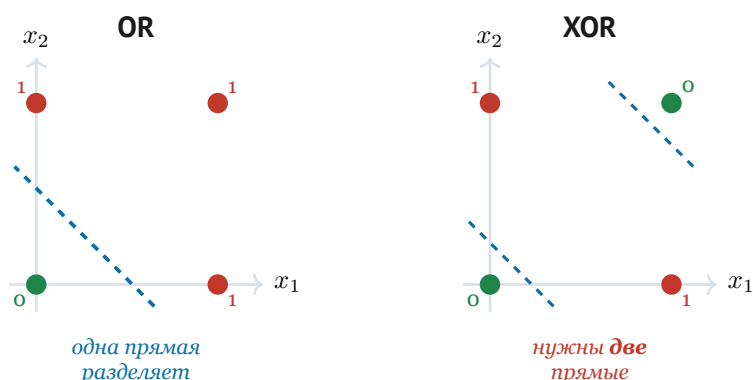
Короткий ответ. При считывании **одним числом** XOR невозможен в принципе (и это честно показано в эксперименте). При **поблочном** считывании петля экран → камера превращается в полноценный второй слой — и XOR появляется.

1. Что такое XOR и почему он «трудный»

Четыре логические функции от двух входов. Нас интересует последний столбец:

x_1	x_2	AND	OR	NAND	XOR
0	0	0	0	1	0
0	1	0	1	1	1
1	0	0	1	1	1
1	1	1	1	0	0

Нарисуем входы как четыре точки на плоскости (x_1, x_2) и раскрасим по ответу. Слева — OR (и любой «обычный» вентиль), справа — XOR.



- **AND, OR, NAND:** единицы (●) и нули (●) разделяются **одной прямой** — это **линейно разделимые** функции.
- **XOR:** единицы стоят по одной диагонали, нули — по другой. **Никакая одна прямая** их не разделит (Минский–Пейперт, 1969).

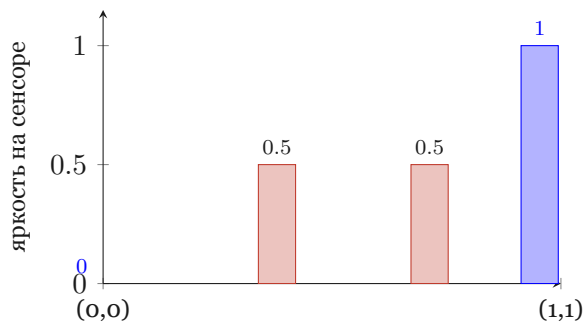
Запомните: «одна прямая» = «один линейный слой» = «одно решающее правило». XOR требует **двух** прямых (двух скрытых нейронов) и ещё одного правила поверх них.

2. Как «Светоч» считает на свету

Сенсор камеры за время экспозиции **складывает падающие фотоны**. Если на экране показать яркости, то яркость на сенсоре — это физическая сумма (а с весами — скалярное произведение). Сложение и скалярное произведение — базовая операция любой нейросети, и здесь она происходит «бесплатно», за один оптический такт. Ключевой вопрос — **как мы это сложение считываем**.

3. Почему текущий XOR-эксперимент честно проваливается

В `stage4_logic.js` выбрана **самая простая** схема считывания: оба входа кодируются **суммарной яркостью** $(x_1 + x_2)/2$ на весь экран, а функция `regionMean` усредняет центральную область кадра (25–75 % по обеим осям) в **один скаляр**.



Коллапс яркости. Вход (x_1, x_2) превращается в одно число $(x_1 + x_2)/2$:

(x_1, x_2)	яркость
(0, 0)	0.0
(0, 1)	0.5
(1, 0)	0.5
(1, 1)	1.0

Случаи **(0, 1) и (1, 0)** дают **одинаковый отсчёт 0.5**. Камера видит «сколько всего света», а не «откуда» — и различить их *не может*.

А ведь XOR требует, чтобы «средние» яркости (0.5) дали **1**, а «крайние» (0.0 и 1.0) — **0**. Это два порога (снизу и сверху), один порог так не умеет. Поэтому эксперимент **специально** объявляет успехом *низкую* точность XOR — провал здесь запланирован и доказывает теорему Минского–Пейперта на реальном железе, а не в симуляции. AND/OR/NAND при этом проходят на 100 %.

Вывод по вопросу №1. Да, в этом эксперименте камера интегрирует по всему (центральному) кадру в одно число. Но это **выбор схемы считывания**, а не предел железа. И при таком считывании обратная связь действительно почти бесполезна — комментатор прав.

4. Вариант есть: поблочное считывание

Камера **не обязана** давать одно число. В `index.html` есть инструменты пространственного показа и считывания:

- `showBlockPattern(values, gridSize)` — показать на экране **сетку** разных яркостей (а не одну заливку);
- `blockMeans(frame, gridSize)` — считать кадр как **сетку регионов** и вернуть массив средних по каждой клетке.



ЭКРАН: сетка яркостей СЕНСОР: массив скаляров

Так уже работают «взрослые» эксперименты — скалярное произведение (`stage2`), `MatVec` (`stage3`) и слой трансформера (`stage5`): разные участки экрана и сенсора **параллельно** считывают разные скалярные произведения — десятки независимых «нейронов» за один кадр. Как только мы умеем считывать **несколько чисел**, у нас появляется несколько скрытых нейронов — ровно то, чего не хватало для XOR.

5. XOR как двухслойная сеть через обратную связь

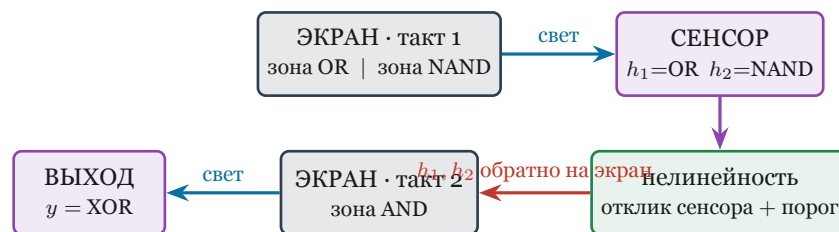
XOR раскладывается в классику:

$$\text{XOR}(x_1, x_2) = \text{AND}(\text{OR}(x_1, x_2), \text{NAND}(x_1, x_2))$$

x_1	x_2	OR	NAND	AND(OR,NAND) = XOR
0	0	0	1	0
0	1	1	1	1
1	0	1	1	1
1	1	1	0	0

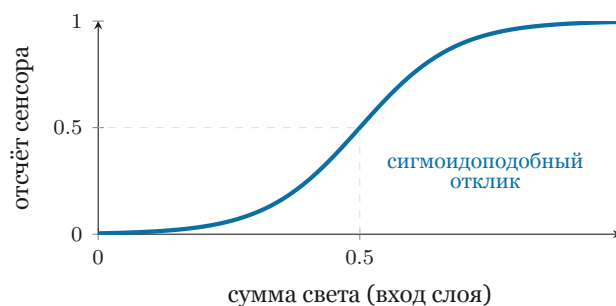
OR и NAND по отдельности линейно разделимы (их «Светоч» уже берёт на 100 %). Значит, нужен **слой 1** (посчитать OR и NAND), **нелинейность** между слоями и **слой 2** (посчитать AND). Оптически — два такта с обратной связью экран → камера.

Архитектура: два оптических такта



Слой 1 — два скрытых нейрона в двух зонах (один такт). Через showBlockPattern показываем входы так, что левая половина экрана настроена на OR, правая — на NAND (разные веса/смещения). Камера через blockMeans считывает два числа h_1, h_2 .

Нелинейность — даётся самим железом. Между слоями обязательно нужна нелинейность (иначе два линейных слоя схлопываются в один). Её не нужно добавлять отдельно — её дают отклик сенсора (гамма + насыщение, кривая сигмоидной формы) и порог $\text{opt} > \text{thresh}$, который в stage4 и так применяется.



Слой 2 — обратная связь (второй такт). Считанные h_1 (OR) и h_2 (NAND) показываем обратно на экране — это и есть петля камера → экран — и настраиваем зону на AND. Камера считывает финальный скаляр; порог по нему даёт 0 для (0, 0) и (1, 1), 1 для (0, 1) и (1, 0). **Это XOR, вычисленный на свету двумя оптическими проходами:** двухслойный перцептрон, развёрнутый во времени.

6. Две формы обратной связи в проекте

- **Пространственно-временная петля** (описана выше): посчитали в регионах → считали → подали назад → ещё слой. Даёт *глубину*.

- **Послесвечение OLED как память** (stage37_lstm): пиксели OLED и тракт «экран→камера» физически удерживают остаточный образ после погасания — ячейка памяти с «забыванием» (forget gate) и постоянной времени τ . Настоящая рекуррентность, где состояние хранится в затухающем свете.
- **Camera-in-the-loop обучение** (статья VI): замыкание петли экран→камера позволяет не только считать, но и *обучать* физический оптический канал, подстраивая паттерны по тому, что увидел сенсор.

«Канал обратной связи», про который спрашивал комментатор, — это не экзотика, а несущая идея: статичный канал считает один слой, а **замкнутый** канал даёт глубину (пространственную) и память (временную).

7. Итоги одним абзацем

- В демо-XOR камера усредняет кадр в **одно число** $\Rightarrow (0, 1)$ и $(1, 0)$ неразличимы $\Rightarrow$ XOR честно проваливается (Минский–Пейперт на железе).
- Это ограничение **схемы считывания**, а не оптики: поблочное считывание даёт **много** скаляров — несколько скрытых нейронов за такт.
- $XOR = AND(OR, NAND)$. OR и NAND берутся слоем 1 в двух зонах; нелинейность даёт сенсор + порог; AND берётся слоем 2.
- **Обратная связь экран → камера** = подать результат слоя 1 обратно на экран для слоя 2. При считывании одним числом — бесполезна; при поблочном — это полноценный второй слой, и XOR появляется.

Примечание. Двухслойная версия — это проектное предложение (следующий эксперимент): в коде stage4_logic.js сейчас один проход с regionMean. Реализация = showBlockPattern на двух зонах + blockMeans для считывания h_1, h_2 + второй проход с подачей h_1, h_2 обратно.